BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Contiguous segment already in ascending order.

Q2. Inserts each next key into its sorted prefix.

Q3. Work scaling with n times log n.

Q4. mid3=7 then mid5=11; return index5.

You Do pages · Independent questions

Q1. After width1 [2,7,1,9] [3]; after width2 [1,2,7,9] [3].

Q2. Compare2/1 choose1 [2], compare2/9 choose2 [1], compare7/9 choose7 [1], append9 [1], result [1,2,7,9] [1].

Q3. Merge O(n log n) [2], typical O(n) extra [1]; insertion O(n^2) worst [2], O(1) extra [1].

Exit Ticket · Model answer

False. It commonly uses O(n) extra storage.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

RHS COMPUTING / OCR H446 / mixed LESSON 081 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

LESSON 081 · Lesson 081 · Merge Sort: Bottom-up Merge Tracing

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What is a sorted run?

Q2. How does insertion sort work?

Q3. What is O(n log n) complexity?

Q4. Binary search sorted [1,3,5,7,9,11,13] for 11: show midpoint index sequence.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

merge sort — Sort by merging sorted runs.

run — Consecutive already sorted segment.

merge — Combine two sorted sequences.

auxiliary space — Extra memory beyond input.

New learning

Observe — Merge sort divides the input into smaller runs; merging two already sorted runs repeatedly selects the smallest available front value.

Apply — Bottom-up merge sort begins with runs of length one and doubles run width on every complete pass; it does not require recursive calls.

Evaluate — Worst-case time O(n log n); standard array merging uses O(n) extra space. Include comparisons, not just the final sorted sequence.

Worked example

Bottom-up merge [8,3,6,1]; show pair results and full result.

1. Method and result — Pairs [3,8] and [1,6] → [3,8,1,6]; merge fronts → [1,3,6,8].

We Do · Practise with guidance

Bottom-up merge [5,2,7,4]; trace first two sorted pairs and final merge.

```
Pair sort [2,5,4,7]; merge sorted runs → [2,4,5,7].
```

Method and result

Pair sort [2,5,4,7]; merge sorted runs → [2,4,5,7].

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Given [7,2,9,1], state array after width1 pair merges and final width2 merge. [6]

Question 2

Merge sorted runs [2,7] and [1,9] and list chosen values in order, with comparisons. [6]

Question 3

Compare bottom-up merge with insertion sort for large random data. Give worst time and auxiliary-space cost. [6]

Exit Ticket · Answer before checking

Merge sort requires no additional array storage in its standard array implementation.